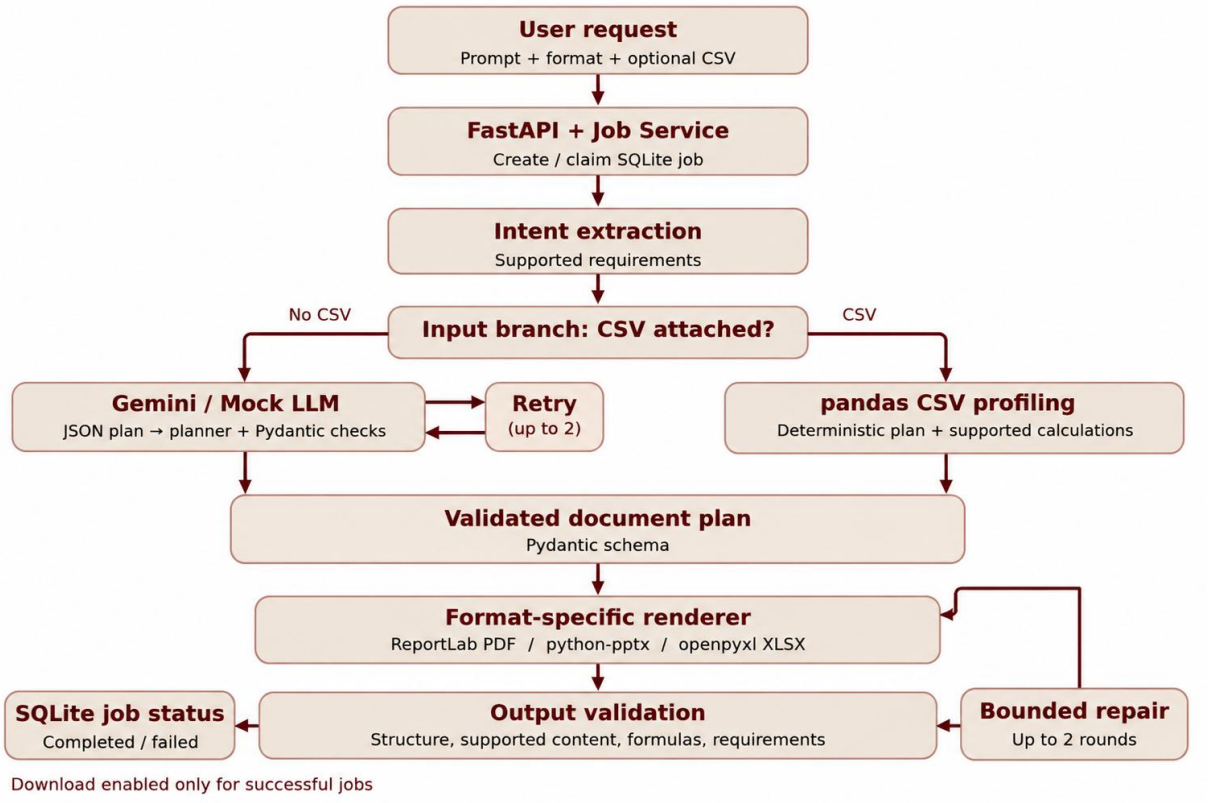
DOXIFY

1. Project Overview

Doxify is a Python application that turns a plain-English request into a downloadable PDF report, PowerPoint presentation, or Excel workbook. The user enters a prompt, chooses a format, and can optionally upload a CSV file. The application prepares a document plan, generates the file, checks supported requirements, and then provides a download link.

2. System Architecture



There are two planning paths. Without CSV input, Gemini or a Mock LLM creates a structured plan. With CSV input, pandas-based Python code profiles the data and creates a plan; the CSV path does not depend on Gemini for the numeric summaries.

Component	Purpose
FastAPI	Receives requests, exposes job status and download endpoints.
SQLite	Stores job information, statuses, errors, and output paths.
Intent + planner	Finds supported prompt requirements and builds a structured plan.

Component	Purpose
Pydantic	Checks the shape and types of the plan.
Renderers	Use ReportLab, python-pptx, and openpyxl to create files.
Validators + repair	Check supported requirements and attempt a limited number of corrections.

3. Important Engineering Decisions

Decision	Reason and trade-off
Structured plan before rendering	Makes format generation predictable; an invalid plan can stop before rendering.
Separate format renderers	Keeps PDF, PPTX, and XLSX code modular, but each format needs its own validation.
pandas for CSV calculations	Reduces dependence on LLM guesses for numbers; custom calculations remain limited.
SQLite for job records	Simple and built into Python; not intended as a large distributed job platform.
Mock LLM fallback	Allows offline and repeatable tests; does not measure real Gemini content quality.
Bounded repair	Prevents endless correction loops; some issues cannot be fixed automatically.

4. Engineering Trade-offs:

Design choice	Benefit	Trade-off
SQLite job store	Simple local database; no separate server.	Not designed for heavy distributed workloads.
Pydantic JSON plans	Clear structure and predictable rendering.	Unusual layouts are harder to represent.
Rule-based validation and repair	Fast and repeatable; no extra AI call needed.	Does not deeply verify facts or meaning.
Mock LLM fallback	Offline and repeatable automated tests.	Does not measure real Gemini output quality.
Predefined Excel formulas	Safer and easier to check.	Cannot handle arbitrary requested formulas.
Local file storage	Easy to build and run locally.	Needs access controls and retention rules for public use.

4. Automated Testing Approach

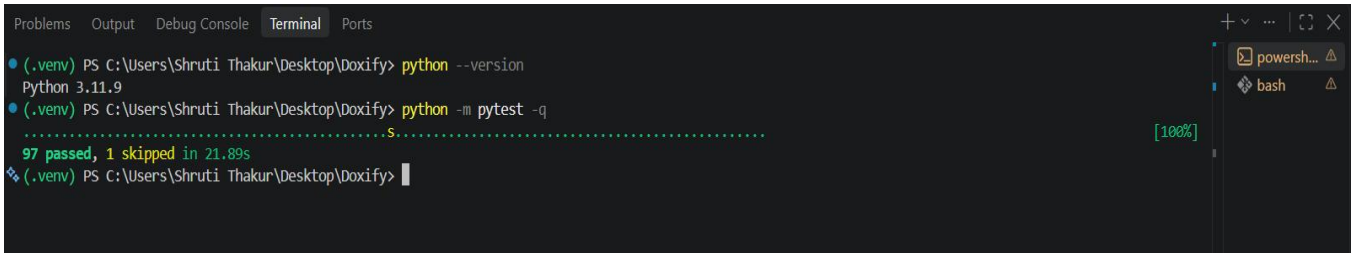
The project uses pytest to check individual modules and selected end-to-end behavior. Tests are written to catch mistakes early and to make failures visible.

Test area	What the tests aim to check
Intent and schemas	Prompt requirements and invalid structured plans.
API and job lifecycle	Job creation, statuses, duplicate requests, and download restrictions.
PDF/PPTX/XLSX rendering	Whether generated files have valid structure and expected elements.
Validation and repair	Whether supported errors are detected and bounded repair is applied.
CSV and numeric rules	Supported calculations and data handling.
Security and limits	Formula injection protections, input constraints, and selected failure paths.

5. Test Result and Environment

Item	Recorded status
Reported pytest result	97 tests passed , 1 skipped
Python environment	Python 3.11 virtual environment
LLM mode used by most automated tests	Mock LLM / deterministic test behavior.
Real Gemini quality	Not established by the 98 tests.
LibreOffice	Optional; independent formula recalculation checks may be skipped when unavailable.
Other environments	Python 3.9 previously caused a ReportLab import error during collection.

Note: A passing pytest run shows that the tested code paths behaved as expected in that environment. It does not prove that every prompt will produce correct facts, ideal formatting, or a good response from a live LLM.



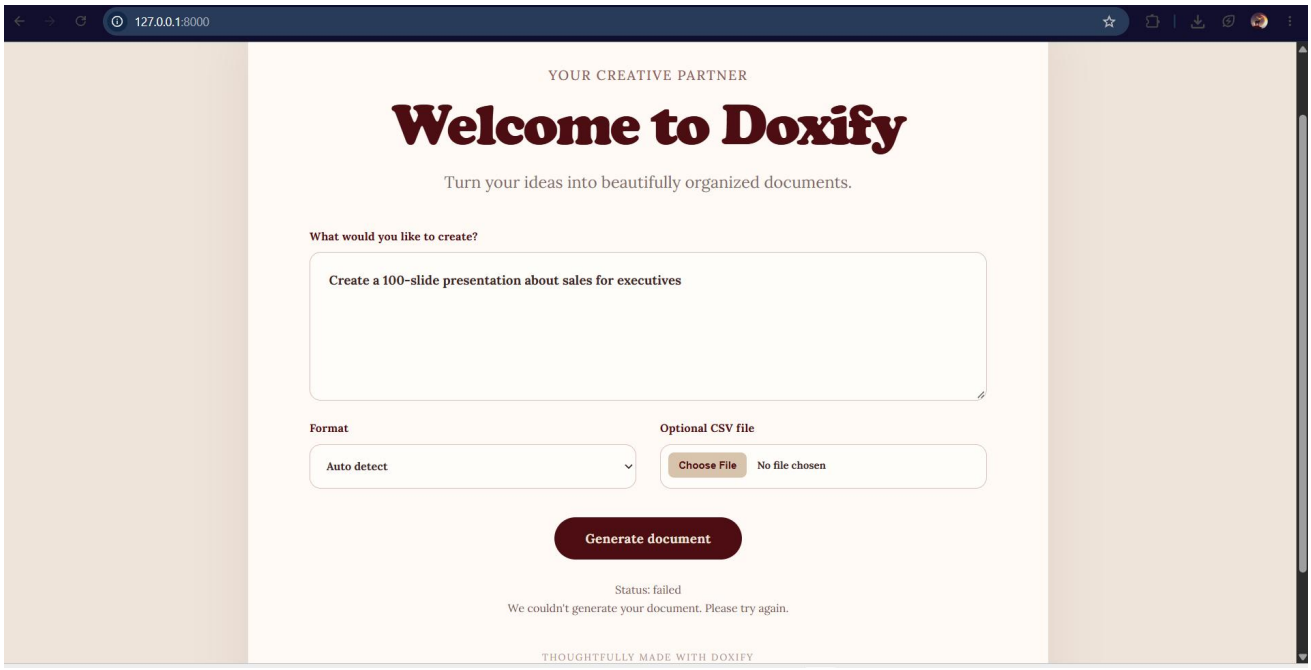
97 test cases passed with 1 skipped (LibreOffice was not installed)

6. Failure Scenario Testing

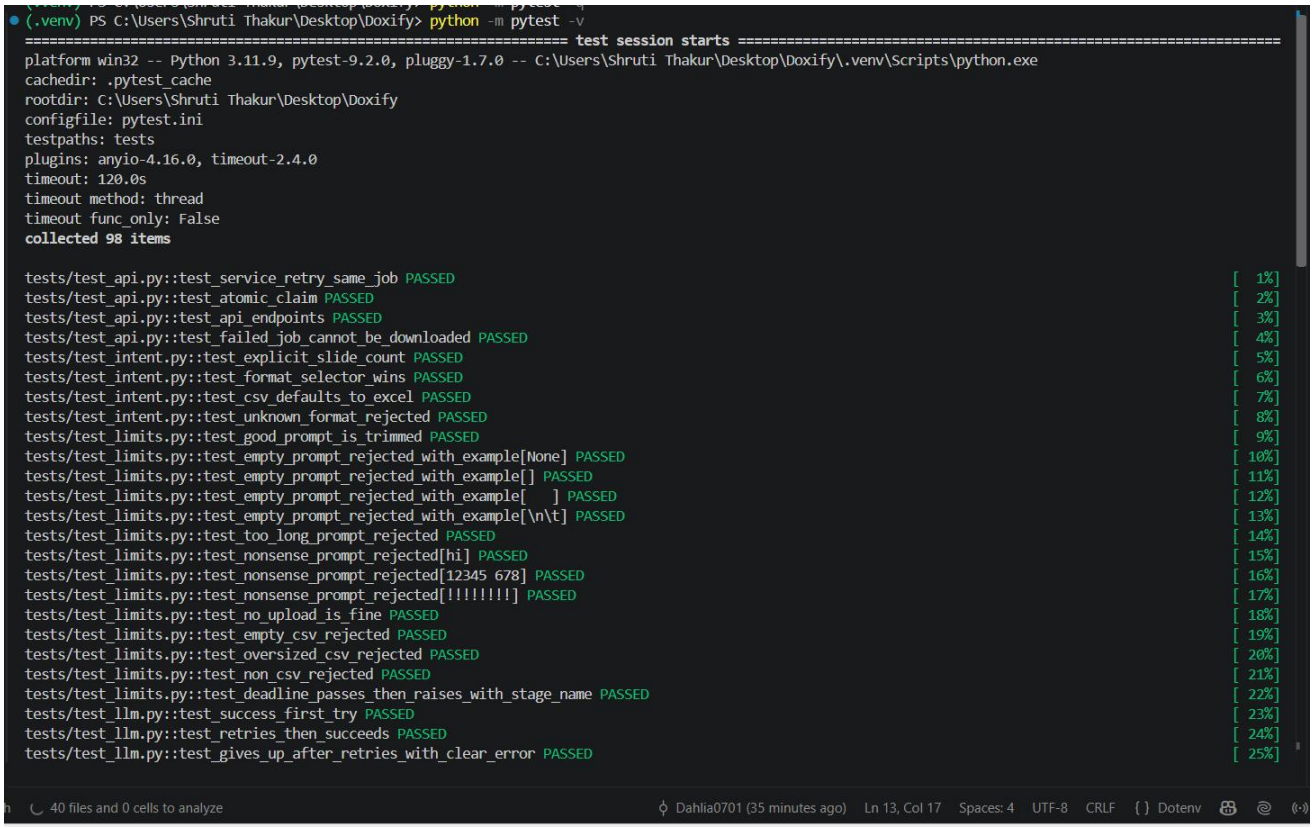
Here application does not show a successful download when generation or validation fails.

Scenario	Expected behavior / evidence
Invalid plan or missing required fields	Reject the plan or mark the job failed rather than render unsupported data.
Malformed or oversized CSV	Return a clear error instead of crashing the application.
LLM error or timeout	Record the failure and avoid reporting success.
Validation failure	Attempt supported repairs within the limit; otherwise report failure.
Failed-job download	Do not expose a downloadable file for a failed job.
Spreadsheet formula injection	Treat unsafe user-supplied formula-like text as text.
Repeated requests	Apply the configured duplicate-job behavior; a failed duplicate may reuse the existing job.

Important: This table describes the expected behavior and reported test areas. It is not a claim that every scenario above has been independently rerun or individually witnessed in this report.



Failure handling example



. Individual test names examples

7. Limitations

Area	Current limitation
Content accuracy	Checks are mainly structural and rule-based. Factual correctness, relevance, and audience suitability are not deeply verified.
CSV formulas	Only recognized columns and predefined calculations are supported.
Layout	Exact PDF page counts may create blank space; overflow checks are limited.
Concurrency	Lightweight local worker setup; heavy parallel use and restart recovery are not proven.
Retries	Failed jobs do not have a general retry workflow; duplicate requests may reuse old jobs.
Security	No user authentication, per-user authorization, or rate limiting for public deployment.

8. Possible Improvements

The next improvements would be:

(1) A separate text-quality checker for factual consistency, relevance, and audience

(2) Safer prompt-driven formula planning

(3) Better PDF pagination and visual overflow checks

(4) More reliable job retries and worker recovery

(5) User accounts and access controls

(6) More integration tests with the real Gemini service.

(7) Allow users to choose themes, fonts, colors, layouts, and images to personalize generated documents.

9. Conclusion

Doxify is an AI-powered document generation application that converts natural-language prompts and CSV data into PDF reports, PowerPoint presentations, and Excel workbooks. Its main advantages are automated document creation, reduced manual effort, structured planning, and built-in validation and repair mechanisms. However, the current system has limitations, including predefined Excel calculations, limited design customization, and the inability to fully verify factual accuracy and content relevance. Despite these limitations, Doxify successfully demonstrates a complete document generation workflow and provides a strong foundation for future improvements in flexibility, reliability, and user experience.